

Программно-аппаратный комплекс «Цифровой двойник технологической линии горячего цинкования»

Техническая документация

Архитектура, математические модели алгоритма
динамического планирования (APS), интеграция
промышленного протокола OPC UA и разработка клиентских
SCADA/HMI интерфейсов

Содержание

1	Введение	2
2	Архитектура программно-аппаратного комплекса	2
3	Технологическая линия горячего цинкования (7 стадий)	4
4	Математическая модель алгоритма APS	6
4.1	Динамическая приоритизация партий	6
4.2	Оценка загруженности оборудования	6
4.3	Суперпозиция расписания и расчёт свободных окон	7
5	Программная реализация ядра (Java / Spring Boot)	8
5.1	Потокобезопасная работа с расписанием (ScheduleService.java)	8
5.2	Симуляция физических параметров стадий (SimulationService.java)	8
5.3	Организация адресного пространства OPC UA (FactoryNamespace.java)	9
6	Аппаратный мост и сбор телеметрии (Python / FastAPI / UART)	11
7	Разработка пользовательских интерфейсов	12
7.1	Графический SCADA-симулятор (PyGame)	12
7.2	Интерактивная HMI-панель (React)	12
8	Оркестрация и развёртывание системы (Launcher.cs)	14
9	Заключение	15

1 Введение

Настоящая техническая документация описывает архитектуру, физико-химическую модель, математические алгоритмы и программную реализацию киберфизического комплекса управления технологической линией горячего цинкования металлоизделий. Проект разработан в рамках концепции Индустрии 4.0 и реализует парадигму Hardware-in-the-Loop (HIL) — аппаратно-программного полунатурного моделирования.

Ключевой задачей комплекса является интеграция многофакторного алгоритма динамического планирования производства (APS — Advanced Planning and Scheduling) в единый распределённый программный контур. Система объединяет физические микроконтроллеры с датчиками, вычислительное серверное ядро на Java Spring Boot с промышленным сервером OPC UA, аппаратно-программный мост телеметрии на Python FastAPI, двухъярусный 2D-симулятор SCADA на PyGame, веб-дашборд оператора на React и мобильный Android-терминал.

2 Архитектура программно-аппаратного комплекса

Комплекс построен по сервис-ориентированной многоуровневой топологии, включающей четыре взаимосвязанных логических уровня: физический (Edge), шлюзовой (Bridge), серверный (Fog/Core) и клиентский (Client/HMI).

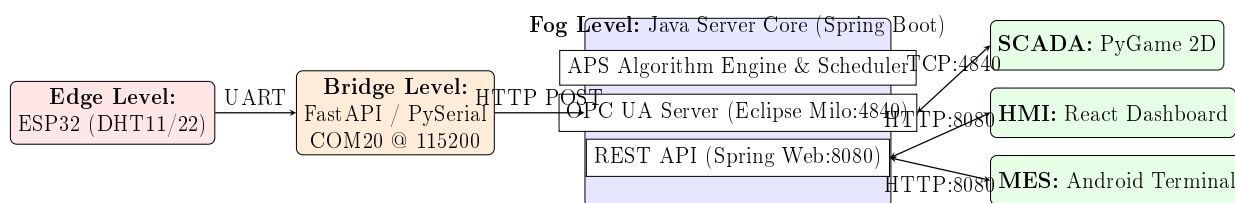


Рис. 1: Структурная схема потоков данных и интерфейсов комплекса

- 1. Аппаратный уровень (Edge):** Физический микроконтроллер ESP32 производит периодический опрос аналогово-цифровых датчиков климатических параметров сушильной камеры и транслирует данные по интерфейсу UART.

2. **Шлюзовой уровень (Bridge):** Модуль `hardware_bridge.py` осуществляет непрерывное чтение COM-порта (115200 бод), первичную фильтрацию пакетов, программную симуляцию смежных агрегатов и фоновую ретрансляцию JSON-пакетов в Java-сервер.
3. **Серверный уровень (Fog):** Высокопроизводительное ядро Spring Boot, управляющее планировщиком задач, хранилищем `schedule.json`, расчётом алгоритма APS и сервером OPC UA.
4. **Уровень человеко-машинного интерфейса (Client):** Включает 2D-симулятор конвейера на PyGame, веб-панель инженера на React и мобильный Android-терминал.

3 Технологическая линия горячего цинкования (7 стадий)

Процесс горячего цинкования представляет собой последовательность взаимосвязанных физико-химических и термических операций, направленных на подготовку поверхности стали и формирование прочного железо-цинкового сплава (Fe – Zn). В соответствии с промышленными регламентами (ГОСТ 9.307-2021) в цифровом двойнике смоделированы 7 технологических стадий:

1. **Стадия 1: Щелочное обезжиривание (1_Alkaline_Degreasing).** Очистка металлоизделий от консервационных масел, смазок и технологических загрязнений в растворе едкого натра (NaOH) и тринатрийфосфата при температуре 50..60°C. Поддержание температуры осуществляется автоматическим включением электронагревателей (ТЭН).
2. **Стадия 2: Промывка №1 (2_Rinsing_1).** Каскадная промывка технической водой для предотвращения уноса щелочного раствора в ванну травления. Включает циркуляционный насос принудительного перемешивания.
3. **Стадия 3: Кислотное травление (3_Pickling).** Удаление прокатной окалины и продуктов коррозии (ржавчины) в растворе ингибированной соляной кислоты (HCl, 120..180 г/л) при температуре 20..30°C. Стадия оснащена барботажным/механическим перемешивателем для интенсификации процесса.
4. **Стадия 4: Промывка №2 (4_Rinsing_2).** Финишная отмывка от остатков кислоты и солей хлорида железа (FeCl₂) с контролем работы промывочного насоса.
5. **Стадия 5: Флюсование (5_Fluxing).** Пассивация и активация поверхности перед контактом с расплавом в водном растворе солей хлорида цинка и аммония (ZnCl₂ · 2NH₄Cl) при температуре 55..65°C с ТЭН-подогревом. Плёнка флюса предотвращает повторное окисление стали при сушке.
6. **Стадия 6: Сушильная камера (6_Drying_Chamber).** Конвекционный нагрев изделий горячим воздухом (50..65°C) для полного испаре-

ния влаги (предотвращение взрывного вскипания воды при погружении в жидкий цинк). Оснащена вентилятором обдува и подключена к физическому датчику DHT11/22 через UART.

7. **Стадия 7: Ванна горячего цинкования (7_Zinc_Bath).** Погружение стальных изделий в расплавленный цинк чистоты 99.995% при температуре 440..460°C. В результате диффузии образуются интерметаллидные фазы гамма, дельта и дзета, обеспечивающие катодную антикоррозионную защиту металла на срок до 50 лет.

Таблица 1: Параметры технологических стадий линии горячего цинкования

Стадия	Среда	Исполнительный орган	Температура
1. Обезжиривание	Щелочь NaOH	ТЭН электронагрева	50..60°C
2. Промывка №1	Вода	Циркуляционный насос	15..25°C
3. Травление	Кислота HCl	Перемешивающее устройство	20..30°C
4. Промывка №2	Вода	Циркуляционный насос	15..25°C
5. Флюсование	Соли ZnCl ₂	ТЭН электроподогрева	55..65°C
6. Сушка	Воздух	Вентилятор обдува, ТЭН	50..65°C
7. Цинкование	Расплав Zn	Высокотемпературный нагрев	440..460°C

4 Математическая модель алгоритма APS

Алгоритм APS решает задачу оптимального динамического диспетчирования партий изделий по многопозиционному агрегату с минимизацией общего времени переналадки и предотвращением технологических конфликтов на оборудовании.

4.1 Динамическая приоритизация партий

Каждая поступающая в очередь MES партия $b \in \mathcal{B}$ характеризуется вектором параметров: временем постановки t_{wait} , маркой сырья m и массой w . Приоритет $P(b)$ формируется аддитивной сверткой:

$$P(b) = U(t_{wait}) + M(m) + W(w) + P_{manual} \quad (1)$$

где:

- $U(t_{wait}) = \min(50, \frac{t_{wait}}{10})$ — штрафная функция за задержку исполнения партии в очереди (баллы от 0 до 50);
- $M(m)$ — технологический вес материала: Медь $\rightarrow 30$, Алюминий $\rightarrow 20$, Сталь $\rightarrow 10$;
- $W(w) = \min(20, \frac{w}{5})$ — весовой балл по массе садки (до 20 баллов);
- $P_{manual} \in \{0, 999\}$ — абсолютный приоритет при ручном вклинивании оператора.

4.2 Оценка загруженности оборудования

Средний коэффициент загруженности производственного комплекса U_{avg} рассчитывается как среднее арифметическое относительных загрузок единиц оборудования:

$$U_{avg} = \frac{1}{N} \sum_{i=1}^N \frac{L_i}{C_i} \quad (2)$$

где L_i — текущая масса обрабатываемых деталей на i -м агрегате (кг), C_i — номинальная вместимость агрегата (кг). Если $U_{avg} \geq U_{crit} = 0,85$, алгоритм временно блокирует включение новых партий во избежание образования заторов на межоперационных накопителях.

4.3 Суперпозиция расписания и расчёт свободных окон

Расчётное время технологической обработки партии на k -й стадии определяется с учётом материального коэффициента K_m и относительной массы заготовки:

$$T_{proc,k} = T_{base,k} \cdot K_m \cdot \left(0,8 + 0,4 \cdot \frac{w}{w_{max}} \right) \quad (3)$$

Для утверждения расписания вычисляется пересечение свободного временного окна оборудования $[\tau_{start}, \tau_{end}]$ с требуемым интервалом обработки T_{proc} . При обнаружении временной коллизии алгоритм переходит в ветку анализа отказа: если число пересчётов $N_{recalc} < 3$, алгоритм производит повторную оптимизацию последовательности действий (коннектор B), иначе переводит систему в режим ручного вмешательства (коннектор C).

5 Программная реализация ядра (Java / Spring Boot)

5.1 Потокбезопасная работа с расписанием (ScheduleService)

Файл `schedule.json` выступает энергонезависимой базой данных задач. Доступ к нему синхронизирован с использованием объекта блокировки `fileLock`:

```
1 @Service
2 public class ScheduleService {
3     private final ObjectMapper mapper = new ObjectMapper();
4     private final File scheduleFile = new File("schedule.json");
5     private final Object fileLock = new Object();
6
7     public ScheduleData readSchedule() {
8         synchronized (fileLock) {
9             if (!scheduleFile.exists()) return new ScheduleData();
10            try {
11                return mapper.readValue(scheduleFile, ScheduleData.class);
12            } catch (IOException e) {
13                return new ScheduleData();
14            }
15        }
16    }
17
18    public void writeSchedule(ScheduleData data) {
19        synchronized (fileLock) {
20            try {
21                mapper.writerWithDefaultPrettyPrinter().writeValue(
22                    scheduleFile, data);
23            } catch (IOException e) {
24                e.printStackTrace();
25            }
26        }
27    }
28 }
```

Листинг 1: Синхронизация доступа к файлу расписания

5.2 Симуляция физических параметров стадий (SimulationS

Фоновый процесс с частотой 0.5 Гц моделирует физические параметры технологических агрегатов:

```

1 @Scheduled(fixedRate = 2000)
2 public void galvanicStagesSimulationLoop() {
3     if (!apiState.isSimRunning()) return;
4
5     // Стадия 1: Обезжиривание (50-60 C)
6     int t1 = 45 + random.nextInt(11);
7     apiState.getStage1().setTemperature(t1);
8     apiState.getStage1().setHeaterOn(t1 < 50);
9     apiState.getStage1().setActive(random.nextBoolean());
10
11     // Стадия 2: Промывка 1 (насос)
12     boolean pump2 = random.nextBoolean();
13     apiState.getStage2().setPumpRunning(pump2);
14     apiState.getStage2().setActive(pump2);
15
16     // Стадия 3: Травление (мешалка)
17     int t3 = 20 + random.nextInt(11);
18     apiState.getStage3().setTemperature(t3);
19     apiState.getStage3().setAgitatorOn(random.nextBoolean());
20
21     // Стадия 7: Ванна расплава цинка (440-460 C)
22     int t7 = 440 + random.nextInt(21);
23     apiState.getStage7().setTemperature(t7);
24     apiState.getStage7().setHeaterOn(t7 < 448);
25     apiState.getStage7().setActive(true);
26 }

```

Листинг 2: Симуляция параметров 7 стадий в SimulationService.java

5.3 Организация адресного пространства OPC UA (Factory)

Класс реализует спецификацию IEC 62541 на базе Eclipse Milo, создавая 8 иерархических папок узлов:

```

1 public void registerNodes() {
2     UByte rwAccess = UByte.valueOf(3); // Чтение и запись
3
4     UaFolderNode folder1 = createFolder("1_Alkaline_Degreasing");
5     stage1TempNode = createVariable(folder1.getNodeId(), "1
6     _Alkaline_Degreasing/Temperature", "Temperature", Identifiers.Int32,
7     new Variant(50), rwAccess);
8     stage1ActiveNode = createVariable(folder1.getNodeId(), "1
9     _Alkaline_Degreasing/IsActive", "IsActive", Identifiers.Boolean, new
10    Variant(false), rwAccess);
11    stage1HeaterNode = createVariable(folder1.getNodeId(), "1
12    _Alkaline_Degreasing/HeaterOn", "HeaterOn", Identifiers.Boolean, new
13    Variant(false), rwAccess);
14 }

```

```

8
9      // Папка сушильной камеры с датчиком UART
10     UaFolderNode folder6 = createFolder("6_Drying_Chamber");
11     stage6TempNode    = createVariable(folder6.getNodeId(), "6
_Drying_Chamber/TemperatureCelsius", "TemperatureCelsius", Identifiers.
Int32, new Variant(55), rwAccess);
12     stage6HumidityNode = createVariable(folder6.getNodeId(), "6
_Drying_Chamber/HumidityPercent", "HumidityPercent", Identifiers.Int32,
new Variant(20), rwAccess);
13     stage6SensorConnectedNode = createVariable(folder6.getNodeId(), "6
_Drying_Chamber/SensorConnected", "SensorConnected", Identifiers.
Boolean, new Variant(true), rwAccess);
14 }

```

Листинг 3: Регистрация узлов в FactoryNamespace.java

6 Аппаратный мост и сбор телеметрии (Python / FastAPI / UART)

Модуль `hardware_bridge.py` организует физический контур сбора данных HIL. Скрипт прослушивает порт COM20 с параметрами 115200 8N1 и при получении пакета обновляет структуру сушильной камеры:

```
1 async def read_serial_data():
2     global factory_state, ser
3     while True:
4         try:
5             if ser and ser.is_open and ser.in_waiting > 0:
6                 line = ser.readline().decode('utf-8', errors='ignore').
strip().lower()
7                 if 'temp:' in line and 'hum:' in line:
8                     parts = line.split(';')
9                     t = int(parts[0].replace('temp:', '').strip())
10                    h = int(parts[1].replace('hum:', '').strip())
11                    node6 = factory_state["stage_6_drying_chamber"]
12                    node6["temperature_celsius"] = t
13                    node6["humidity_percent"] = h
14                    node6["sensor_connected"] = True
15                except Exception:
16                    factory_state["stage_6_drying_chamber"]["sensor_connected"] =
False
17                    await asyncio.sleep(0.1)
18
19 async def forward_to_java_server():
20     while True:
21         try:
22             url = "http://localhost:8080/api/factory-status"
23             payload = json.dumps({"status": "ok", "data": factory_state}).
encode('utf-8')
24             req = urllib.request.Request(url, data=payload, headers={'
Content-Type': 'application/json'}, method='POST')
25             urllib.request.urlopen(req, timeout=1)
26         except Exception:
27             pass
28         await asyncio.sleep(1)
```

Листинг 4: Чтение UART и ретрансляция телеметрии в `hardware_bridge.py`

7 Разработка пользовательских интерфейсов

7.1 Графический SCADA-симулятор (PyGame)

Клиент визуализирует перемещение заготовок по двум технологическим линиям с независимыми мостовыми манипуляторами. Логика манипулятора построена на конечном автомате:

```
1  # Приоритет 1: Выгрузка готовой детали с последней станции
2  last_station = self.stations[-1]
3  if last_station.current_part and last_station.current_part.get('
    ready_for_next'):
4      self.crane_state = "MOVE_TO_PICKUP"
5      self.crane_target = last_station.current_part
6      self.pickup_x = last_station.x
7      self.drop_x = self.x_end
8
9  # Приоритет 2: Перемещение между ваннами от конца к началу
10 for i in range(len(self.stations) - 2, -1, -1):
11     src_st = self.stations[i]
12     dst_st = self.stations[i + 1]
13     if src_st.current_part and src_st.current_part.get('ready_for_next')
    and not dst_st.current_part:
14         self.crane_state = "MOVE_TO_PICKUP"
15         self.pickup_x = src_st.x
16         self.drop_x = dst_st.x
17         break
```

Листинг 5: Конечный автомат манипулятора в pygame-simulator/main.py

7.2 Интерактивная HMI-панель (React)

Веб-интерфейс обеспечивает мониторинг выполнения фаз блок-схемы алгоритма APS, отображение телеметрических карточек 7 стадий и оперативное управление очередью заказов.

```
1  useEffect(() => {
2      const pollFactoryState = async () => {
3          try {
4              const res = await fetch('http://localhost:8080/api/factory-status');
5              const json = await res.json();
6              if (json.status === 'ok') setFactoryStages(json.data);
7          } catch (e) {
8              console.warn('Сервер недоступен:', e);
9          }
10     }
```

```
10   };  
11   const timer = setInterval(pollFactoryState, 500);  
12   return () => clearInterval(timer);  
13 }, []);
```

Листинг 6: Динамический поллинг метрик и стадий в React дашборде

8 Оркестрация и развёртывание системы (Launcher)

Для автономного запуска комплекса без необходимости ручного вызова отдельных терминальных команд разработан модуль `Launcher.cs`. Оркестратор запускает ядро Java, проверяет доступность TCP-порта 8080, открывает браузер по умолчанию, стартует PyGame-симулятор и регистрирует перехватчики сигналов завершения Win32:

```
1 static void Main(string[] args) {
2     SetConsoleCtrlHandler((sig) => { Cleanup(); return true; }, true);
3     AppDomain.CurrentDomain.ProcessExit += (s, e) => Cleanup();
4
5     string currentDir = AppDomain.CurrentDomain.BaseDirectory;
6     string jarPath = Path.Combine(currentDir, "server.jar");
7     string simPath = Path.Combine(currentDir, "simulator.exe");
8
9     serverProcess = Process.Start(new ProcessStartInfo {
10         FileName = "java", Arguments = "-jar \"" + jarPath + "\"",
11         WorkingDirectory = currentDir, UseShellExecute = false
12     });
13
14     int attempts = 0;
15     while (attempts++ < 25) {
16         if (CheckPort(8080)) break;
17         Thread.Sleep(1000);
18     }
19
20     Process.Start("http://localhost:8080");
21     if (File.Exists(simPath)) {
22         simulatorProcess = Process.Start(new ProcessStartInfo { FileName =
23             simPath, WorkingDirectory = currentDir });
24     }
```

Листинг 7: Координация процессов в `Launcher.cs`

9 Заключение

Разработанный программно-аппаратный комплекс цифрового двойника технологической линии горячего цинкования подтверждает практическую применимость методов динамического планирования APS в условиях реального промышленного производства.

Интеграция протокола OPC UA позволила организовать детерминированный обмен технологическими данными между сервером и исполнительными механизмами, а аппаратно-программный мост HIL обеспечил верификацию алгоритмов планирования на реальных физических сигналах микроконтроллеров. Модульная архитектура гарантирует масштабируемость системы и возможность адаптации под смежные гальванические и металлургические производства.